HPC Project Report 2025

Ponesch, Techen, Olsacher

June 2, 2025

1 Introduction

This project report presents the implementation and evaluation of an allgather-merge operation for integers, developed as part of the course *184.725 High Performance Computing* at *TU Wien*. The objective was to design efficient parallel algorithms using MPI in C++, beginning with a baseline implementation based on `MPI_Allgather` followed by sequential merging.

Subsequent improvements were made by incorporating graph-based communication strategies, specifically the **Bruck straight doubling circulant graph algorithm** and a **circulant graph allreduce operation** where merging replaces the standard reduction operation. In addition to algorithmic enhancements, optimizations such as pointer-based data handling, reduced local copy operations, and minimized memory allocations were explored to further improve performance and memory efficiency.

Benchmarking was conducted using the provided `main.cpp` file, facilitated by a `CMakeLists.txt` build system for streamlined compilation and testing. To execute the performance evaluations on the Hydra cluster [2], the supplied `job-run.sh` script was adapted to accommodate varying node counts, as required by the benchmarking tasks.

2 Task Description

A concise description of the tasks to solve in this project is given in the Assignment, handed out by the lecturers [1], but to include a brief overview over the goals of this project, this section was included in the report. The **integer allgather-merge problem** is defined as the following:

> "Given p input blocks, one for each of the p processes, each with the same number m/p of integer elements, each block in sorted order, for a total of m input elements. The output for each process is a single block of the m input elements but in sorted order. Duplicates should not be removed, and no duplicates introduced." [1]

To implement a solution for this problem, three different approaches had to be implemented, the Baseline Algorithm (Exercise 1), the Bruck Algorithm (Exercise 2) and the Circulant All Reduce Algorithm (Exercise 3). The three algorithms are then benchmarked according to Exercise 4.

2.1 Exercise 1 – Baseline Algorithm

The baseline implementation uses the standard `MPI_Allgather` operation to collect all sorted input blocks at each process. After the gathering step, each process independently performs a sequential p-way merge of the collected blocks to produce the final sorted output. The implementation aims to minimize overhead by exploring efficient merging strategies and possible optimization. The Baseline Algorithm is briefly outlined in Algorithm 1.

Algorithm 1 Baseline Allgather-Merge Algorithm

Require: Sorted local array R_r of size m/p at each process r
Ensure: Sorted global array M_r of size m at each process r
1: **Allgather** all local arrays R_r to form $G_r = \{R_0, R_1, \ldots, R_{p-1}\}$ using `MPI_Allgather`
2: **Merge** the p sorted blocks in G_r into a single sorted array M_r using a p-way merge
3: **return** M_r

2.2 Exercise 2 – Bruck Algorithm

This exercise requires improving the baseline by integrating the merging process into the communication phase. Based on the Bruck straight doubling circulant graph algorithm, merging is performed progressively after each communication round, reducing the total number of elements merged at once and potentially lowering memory and communication overheads. Special attention is needed to handle non-power-of-two process counts and to minimize local copy operations. The Bruck Algorithm is briefly outlined in Algorithm 2.

Algorithm 2 Bruck Allgather Algorithm

Require: Local block $R[r]$ of size m/p at process r
Ensure: All blocks gathered at each process
1: $q \leftarrow \lceil \log_2 p \rceil$
2: $B[0] \leftarrow R[r]$
3: **for** $k = 0$ to $q - 1$ **do**
4: $s_k \leftarrow 2^k$
5: $t \leftarrow (r - s_k + p) \bmod p$
6: $f \leftarrow (r + s_k) \bmod p$
7: Send $B[0 \,..\, s_k - 1]$ to process t
8: Receive $B[s_k \,..\, 2s_k - 1]$ from process f
9: **end for**
10: **for** $i = 1$ to $p - 1$ **do**
11: $R[(r + i) \bmod p] \leftarrow B[i]$
12: **end for**

2.3 Exercise 3 – Circulant All Reduce Algorithm

Building on Exercise 2, this variant adapts the circulant graph allreduce algorithm by replacing the reduction operation with a two-way merge. Unlike Bruck's algorithm, it aims to balance communication more evenly across processes and reduce the number of rounds, further improving scalability. The focus is on achieving efficient distributed merging with minimal redundant computation and memory usage. The Circulant Algorithm is briefly outlined in Algorithm 3.

Algorithm 3 Circulant Allreduce Algorithm

Require: Input vector V, output vector W, process rank r in C_p^s
Ensure: Each process computes reduction of all input vectors

if $p = 1$ **then**
  $W \leftarrow V$
  return
end if
for $k = 0$ to $q - 1$ **do**
  $\epsilon \leftarrow s_{k+1} \wedge 0x1$ ▷ Adjustment term: 1 if s_{k+1} is odd, 0 otherwise
  $t \leftarrow (r - s_k + \epsilon + p) \mod p$
  $f \leftarrow (r + s_k - \epsilon) \mod p$
  if $\epsilon = 1$ **then**
    Send W to t **parallel with** Receive T from f
    $W \leftarrow W \oplus T$
  else
    if $k = 0$ **then**
      Send V to t **parallel with** Receive W from f
    else
      $W' \leftarrow V \oplus W$
      Send W' to t **parallel with** Receive T from f
      $W \leftarrow W \oplus T$
    end if
  end if
end for
$W \leftarrow V \oplus W$

2.4 Exercise 4 – Benchmarks

All three implementations are benchmarked across various input sizes and process configurations. Benchmarks are conducted on the Hydra cluster [2], using weak scaling scenarios. Execution times are averaged over multiple runs, and the results are analyzed to compare the performance and scalability of the different approaches. The process configuration to benchmark are:

- 1 Node × 1 Process per Node = 1 Total Processes
- 1 Node × 10 Process per Node = 10 Total Processes
- 1 Node × 32 Process per Node = 32 Total Processes
- 10 Node × 1 Process per Node = 10 Total Processes
- 10 Node × 10 Process per Node = 100 Total Processes
- 10 Node × 32 Process per Node = 320 Total Processes
- 20 Node × 1 Process per Node = 20 Total Processes

- 20 Node $\times$ 10 Process per Node = 200 Total Process
- 20 Node $\times$ 32 Process per Node = 640 Total Processes

and the different starting block sizes for each process are m/p = 1, 10, 100, 1 000, 10 000, 100 000.

3 Implementations

The three required algorithms were developed within the provided project framework, which included a set of predefined files intended to support compilation, benchmarking, and testing. Specifically, the assignment included:

- `CMakeLists.txt`: Configuration file to facilitate building both debug and release versions of the binaries.
- `job-run.sh`: Script for queuing benchmark jobs on Hydra using `sbatch`, allowing easy specification of node counts (e.g., 1, 10, or 20 nodes).
- `main.cpp`: Manages benchmark setup, including input generation, time measurement, MPI initialization, and invoking the implemented algorithms.
- `algorithms.h`: Header file that declares the interfaces of the algorithm implementations.
- `baseline.cpp`: Implementation file for the Baseline Algorithm.
- `algorithm1.cpp`: Implementation file for the Bruck Algorithm.
- `algorithm2.cpp`: Implementation file for the Circulant Allreduce Algorithm.

All algorithm implementations were first developed and validated locally before being deployed on the Hydra cluster. To gain deeper insights into performance characteristics, the **profiling** and **tracing** tool *HPCToolkit*, available via `spack` on Hydra, was utilized. In particular, *HPCViewer* was used during the development of the Bruck Algorithm to analyze execution traces and identify potential optimization opportunities.

3.1 Baseline Algorithm

The Baseline Algorithm implementation first utilizes the `MPI_Allgather` operation to collect all locally sorted integer blocks from every process onto every process. Following the gathering phase, each process independently performs a **direct p-way merge** of the received blocks. The merge is implemented using a `priority_queue` combined with `HeapNode` structures to efficiently manage and extract the minimum element at each step [3]. The implementation is briefly outlined in Algorithm 4.

Algorithm 4 Baseline Allgather-Merge Implementation

Require: Locally sorted array at each process, $sendbuf$ of size $sendcount$
Ensure: Globally sorted array in $recvbuf$ of size $size \times recvcount$

Get process rank and size: $rank \leftarrow$ `MPI_Comm_rank`, $size \leftarrow$ `MPI_Comm_size`
Perform `MPI_Allgather` to collect all local arrays into $recvbuf$
Initialize a min-heap H based on a custom comparator (HeapNode: process index, position, pointer to value)
Initialize $sources[i]$ pointers to the beginning of each process's block in $recvbuf$
for each process i from 0 to $size - 1$ **do**
    if $recvcount > 0$ **then**
        Push $(i, 0, sources[i])$ into the heap H
    end if
end for
Initialize $merged$ array of size $size \times recvcount$
$merged\_offset \leftarrow 0$
while heap H is not empty **do**
    Pop the top node (idx, pos, ptr) from heap
    $merged[merged\_offset] \leftarrow *ptr$
    Increment $merged\_offset$
    if there are more elements in the current block **then**
        Update $pos \leftarrow pos + 1$
        Update $ptr \leftarrow sources[idx] + pos$
        Push updated node back into heap
    end if
end while
Copy the merged array back into $recvbuf$
return `MPI_SUCCESS`

Asymptotic Complexity and Memory Usage The baseline algorithm first executes an `MPI_Allgather`, which under the assignment assumptions costs $\mathcal{O}(m + \log p)$ operations per process. After gathering, each process performs a p-way merge using a binary heap of size p. Initial heap construction costs $\mathcal{O}(p \log p)$, and the merge loop performs m `pop-push` operations, each taking $\mathcal{O}(\log p)$ time, leading to an overall computation time of $\mathcal{O}(m \log p)$ per process. Since each process performs the full merge independently, the aggregate computational work is $\mathcal{O}(pm \log p)$. The algorithm uses an auxiliary buffer of size m for the merged result in addition to the receive buffer, leading to $\mathcal{O}(m)$ peak memory usage per process. Each element is read from `recvbuf`, written to the auxiliary buffer, and copied back once, resulting in exactly two full-length memory copies, i.e., $\Theta(m)$ additional memory movements beyond the network communication.

3.2 Bruck Algorithm

The Bruck Algorithm only covers the definition of the communication patterns between the processes, what is done between the communications, or after, or even what is send or received is up to our implementation. For this we started out with Version 1, which was the easiest to implement and gradually improved the implementation by using *HPCToolkit*.

3.2.1 Bruck – Version 1

The first version of the Bruck Algorithm leverages the Bruck straight doubling circulant graph communication pattern to distribute sorted integer blocks across all processes. This method provides a more communication-efficient alternative to the traditional `MPI_Allgather`.

To optimize memory usage and reduce overhead, the implementation favors pointer arithmetic to avoid unnecessary memory copies. Communication is performed using `MPI_Sendrecv`, where in each round, blocks of data are sent and received based on the skip distances defined by the Bruck algorithm. The data blocks are received directly into the pre-allocated `recvbuf`, which is managed externally by `main.cpp`, minimizing dynamic memory allocation and additional buffer overhead.

Importantly, the data exchanged during the communication rounds remains unsorted. Only after the communication phase is complete, a **direct p-way merge** is performed to produce the final sorted array. The p-way merge is implemented identically to the approach used in the Baseline Algorithm [3].

Although this initial version retains the sequential merging phase similar to the baseline, it already reduces communication redundancy and intermediate copying, serving as a foundation for further optimization.

3.2.2 Bruck – Version 2

The second version of the Bruck Algorithm further refines the communication and merging strategy. As in the first version, the Bruck straight doubling circulant graph pattern is used to orchestrate communication between processes. However, instead of gathering all blocks first and performing a single global merge at the end, this version incrementally merges after each communication round.

In each round, processes exchange unsorted blocks using `MPI_Sendrecv`, with data sizes and partners determined by the Bruck communication pattern. After each exchange, the newly received blocks are immediately merged with the existing local data. Depending on the number of received blocks, either a simple two-way merge (`std::merge`) or a **p-way merge** is performed, where k is determined by the number of blocks received in the current round [3].

To avoid unnecessary memory copying, two buffers are maintained and swapped after each merge, allowing the merge results to be written without additional allocations. Only at the end, if needed, the final merged data is copied back into the designated `recvbuf`.

While this approach reduces the size of data that needs to be merged in each round compared to a full post-communication merge, it was eventually discontinued in favor of a more optimized final version due to performance considerations. At this point, profiling with *HPCToolkit* revealed that the main bottleneck in this approach was the inefficiency of the ***p*-way merge** step, which dominated the total execution time despite the reduced communication overhead.

3.2.3 Bruck – Version 3

The third version of the Bruck Algorithm focuses on reducing the cost of merging by minimizing the number of expensive ***p*-way merges**, even at the expense of increased communication volume. It continues to use the Bruck straight doubling circulant graph pattern to orchestrate communication, but adopts a new hybrid data exchange strategy.

In each communication round, every process sends a combination of its locally merged sorted data and its unmerged blocks. The merged blocks enable efficient two-way merges after each communication round, while the unmerged blocks are transmitted to ensure that in the final step, only the missing data blocks of the communiaction partner need to be exchanged. These unmerged blocks are stored directly in the `recvbuf` in a structured order, so they can be correctly used in the last round without additional reorganization.

While this strategy increases the size of the messages being exchanged – thus increasing communication time – it significantly reduces the need for costly multi-way merges, but also increases the memory buffers allocated. After each `MPI_Sendrecv`, the merged data is combined with the received merged blocks via a simple two-way merge. Only in the final round, if multiple unmerged blocks remain, a ***p*-way merge** is performed.

This design shifts the performance balance: larger communication buffers lead to longer messages, but substantially lower merge overhead compared to earlier versions. Profiling with *HPCToolkit* confirmed that this trade-off results in notable improvements in total execution time. However, the profiling results also revealed that despite the optimizations, the ***p*-way merge step remains the dominant contributor** to the overall execution time in benchmark runs.

3.2.4 Bruck – Final Version

The goal of the final version was to completely eliminate the need for the expensive ***p*-way merge** and instead rely solely on efficient two-way merges. While further optimizations could have been considered, it was deemed that the improvements achieved by this design were sufficient for the scope of this project.

This version continues to use the Bruck straight doubling circulant graph pattern for orchestrating communication between processes. However, unlike previous approaches, it ensures that only fully sorted arrays are exchanged at every communication step. Each process sends its currently merged and sorted

data array after each round and receives a similarly sorted array from its communication partner.

To enable this strategy, a custom structure, `TrackedElement`, was introduced. Each `TrackedElement` stores not only the value of the data element but also the origin – the rank of the process from which the element initially originated – along with a comparison operator to maintain sorting. A custom `merge_with_tracking` function was implemented to perform efficient two-way merges between sorted arrays of `TrackedElement` objects, merging both the data and maintaining the correct origin information throughout the process. Although the use of such a struct introduces some overhead – as each element now carries additional metadata – it was a pragmatic and sufficiently efficient solution for the purposes of this project.

After each `MPI_Sendrecv` operation, the received sorted array is merged with the local sorted array using the two-way merge with tracking. Two preallocated buffers are swapped after each merge step to avoid dynamic memory allocation and minimize copying. The origin information is crucial during the final round, where each process needs to selectively send only the missing elements required by its partner, based on their original source ranks. This enables a final selective data exchange without transmitting the entire merged array.

The main trade-off in this approach is the slight increase in memory and computational overhead due to origin tracking, but it completely removes the need for any **p-way merge** – even in the last round. All merge operations are performed as efficient two-way merges, leading to a significant reduction in computational complexity and memory management overhead compared to previous versions.

It is worth noting that the two-way merge used in this implementation, while effective, is not theoretically optimal. Further improvements could be achieved by replacing the standard merge procedure with more advanced techniques such as **Loser Trees** [4], which are known to provide better performance for multi-way merging tasks.

In conclusion, this final version achieves the intended goal: eliminating costly p-way merges, ensuring that every merge is a two-way merge, and keeping the data exchanges fully sorted throughout the process. Profiling results confirmed that this led to substantial performance improvements, bringing the merging phase to an efficiently scalable level. Algorithm 5 briefly outlines how the algorithm was implemented.

Algorithm 5 Final Bruck Allgather-Merge Algorithm with Two-Way Merges and Origin Tracking

Require: Sorted local array $R[r]$ of size m/p at process r
Ensure: Fully sorted array of size m at each process
1: Initialize $curr \leftarrow$ array of $(value, origin)$ pairs from local data with $origin = r$
2: Initialize empty buffer $next$ for merging
3: $q \leftarrow \lceil \log_2 p \rceil$
4: **for** $k = 0$ to $q - 1$ **do**
5: $s_k \leftarrow 2^k$
6: $t \leftarrow (r - s_k + p) \mod p$ ▷ Target process to send to
7: $f \leftarrow (r + s_k) \mod p$ ▷ Source process to receive from
8: **if** $k == q - 1$ **then**
9: Adjust $s_k \leftarrow s_k - (2^q - p)$
10: Select elements in $curr$ where $(origin - r + p) \mod p < s_k$ for sending
11: $sendptr \leftarrow$ filtered elements
12: **else**
13: $sendptr \leftarrow curr$
14: **end if**
15: `MPI_Sendrecv` $sendptr$ to t, receive from f into $recvbuffer$
16: **Merge** $curr$ and $recvbuffer$ into $next$, keeping sorted order and origin tracking
17: Update $curr \leftarrow next$ for next round (swap buffers)
18: **end for**
19: Extract only the values from $curr$ and write to output buffer
20: **return** sorted array

Asymptotic Complexity and Memory Usage The Bruck Allgather-Merge algorithm performs $\lceil \log_2 p \rceil$ communication rounds, where p is the number of processes. In each round, every process exchanges increasingly large, sorted segments of data with a partner and subsequently merges the received data while tracking origins. The size of merged data doubles in each round, reaching up to m elements, where m is the global number of elements. Consequently, each process performs $\mathcal{O}(m)$ total merge operations over all rounds, leading to an aggregate computational work of $\mathcal{O}(m \log p)$ across all processes. The peak memory usage per process is $\mathcal{O}(m)$, as each process eventually maintains buffers for the entire dataset. Due to the sequential merging, the total number of element-wise memory copies per process is $\mathcal{O}(m \log p)$. Overall, this algorithm achieves asymptotically optimal communication and merge complexity for parallel allgather-based sorting, with practical performance mainly determined by buffer management and MPI communication efficiency.

3.3 Circulant All Reduce Algorithm

We implemented the function very closely to the pseudocode as described in Algorithmus 3, but changed the $\oplus$ with a three-argument merge-subroutine, the two-way merge. It was also given not to merge into the output vector W, but to use a merging buffer M, where M' is only required due to the inability to merge M into itself. The adapted pseudocode is stated as follows:

Algorithm 6 Circulant Allgather Merge Algorithm

Require: Input vector V, output vector W, process rank r in C_p^s
Ensure: Each process merges sorted input vector into output vector

1: **if** $p = 1$ **then**
2: $W \leftarrow V$ **return** ▷ local copy is needed
3: **end if**
4: **for** $k = 0$ to $q - 1$ **do**
5: $\epsilon \leftarrow s_{k+1} \wedge 0x1$ ▷ Adjustment term: 1 if s_{k+1} is odd, 0 otherwise
6: $t, f \leftarrow (r - s_k + \epsilon + p) \mod p, (r + s_k - \epsilon) \mod p$
7: **if** $\epsilon = 1$ **then**
8: **Send**(M, t, C_p^s) $\parallel$ **Recv**(T, f, C_p^s)
9: MERGE(M, T, M')
10: $M \leftarrow M'$ ▷ local copy is needed
11: **else**
12: **if** $k = 0$ **then**
13: **Send**(V, t, C_p^s) $\parallel$ **Recv**(T, f, C_p^s)
14: $M \leftarrow T$ ▷ local copy is needed
15: **else**
16: MERGE(V, M, M')
17: **Send**(M', t, C_p^s) $\parallel$ **Recv**(T, f, C_p^s)
18: MERGE(M, T, M')
19: $M \leftarrow M'$ ▷ local copy is needed
20: **end if**
21: **end if**
22: **end for**
23: MERGE(V, M, W) ▷ local copy is needed, but merges directly into recvbuf

During the development process, we continuously improved our performance in terms of memory allocation and runtime by reducing the number of variables and overhead by using `std::vector::reserve`, dynamically resizing the vectors M, T, M' and handling edge cases(e.g. $p = 1$ or $k = 0$).

Asymptotic Complexity and Memory Usage In the *Circulant Allgather Merge Algorithm*, local copying occurs at several stages where intermediate arrays are reassigned or merged. First, in the special case when $p = 1$, the input vector V is copied directly to the output vector W (Line 2). In the general case, local copying is performed when the temporary received array T replaces the

current merge buffer M (Line 14), and after each merge step, where the result M' is copied back into M (Lines 10 and 19). The final output is produced by merging the original vector V with the accumulated result M into W, which also constitutes a local copy (Line 23). These steps ensure that the buffers used for communication and merging remain separate to preserve data correctness.

Regarding asymptotic complexity, each process performs $\mathcal{O}(\log p)$ communication rounds, as defined by the skip sequence s_k with $q = \lceil \log_2 p \rceil$. In each round, the size of the merged data approximately doubles, leading to an overall merge and copy cost of $\mathcal{O}(m)$ per process, where m is the total number of output elements per processes. The communication cost also amounts to $\mathcal{O}(m)$ in total, assuming point-to-point communication of increasing block sizes. Therefore, the total asymptotic time complexity per process is $\mathcal{O}(m + \log p)$, with the merge and copy steps dominating the local computation cost. The algorithm is space-efficient up to a constant factor, requiring additional buffers of size $\mathcal{O}(m)$ for temporary merge and receive storage. Compared to the implementation of Algorithm 1, where we have a complexity of $\mathcal{O}(m \log p)$, we decrease the number of copies for larger p, therefore as well the runtime, and even more compared to the baseline complexity of $\mathcal{O}(pm \log p)$. We can perfectly observe this behavior in the benchmarking results for $n = 32$ tasks per node and the large message size $m \geq 1000$.

3.4 Execution of Benchmarks

After the final versions of the algorithms were developed, they were benchmarked on the Hydra compute cluster [2]. Three job scripts were prepared to automate the benchmarking process: `job-run_1.sh`, `job-run_10.sh`, and `job-run_20.sh`. The script `job-run_1.sh` executes the program on a single node, running with 1, 10, and 32 processes on that node. Similarly, `job-run_10.sh` and `job-run_20.sh` are configured to run the program on 10 and 20 nodes, respectively, again using 1, 10, and 32 processes per node. All benchmark results are displayed in the Tables 1 to 9.

All scripts were submitted to the cluster using the `sbatch` scheduler. The full workflow on Hydra involves uploading the necessary files or directories via `scp`, loading required packages through `spack load`, and configuring the build using `cmake`. Once compiled, the job scripts are submitted with `sbatch`. Upon job completion, the SLURM output files and command-line logs are saved in the respective execution directory, and can be downloaded back to the local machine using `scp`. Detailed instructions for interacting with Hydra can be found in the official documentation.

In the `main.cpp` file, the benchmarking parameters were set to perform 2 warm-up rounds, followed by 10 measurement runs for each configuration. The main program computes the average runtime over these 10 runs, providing a robust estimate of the performance for each tested configuration.

4 Results

The `main.cpp` program outputs the averaged runtimes for each configuration and algorithm in a format compatible with LaTeX tables. These outputs are saved by SLURM into the execution directory and were subsequently collected and integrated into the complete tables presented in this report. The following three subsections each contain three tables, summarizing the results for 1, 10, and 32 processes per node, respectively.

Although the total run time rises sharply when moving from a single node to 10 and 20 nodes, this must be viewed through the lens of *weak scaling*: the global problem size grows linearly with the number of processes, so each larger job manipulates proportionally more data. An ideal allgather–merge would therefore keep the *time per local element* roughly constant, increasing only with the theoretical $O(\log p)$ communication depth. Our results show that none of the three implementations attains this ideal, but the gap varies dramatically.

The **Baseline** algorithm—forced to collect *all* m elements on every process and perform a full p-way merge—incurs both $\Theta(m)$ network traffic and $\Theta(m \log p)$ local work, so its per-element time grows *faster* than linearly in p. **Bruck** trims the excess by merging incrementally, reducing both message volume and merge fan-in; its per-element time still rises, but at roughly half the Baseline rate. **Circulant**, which exchanges only already-sorted halves and performs exclusively two-way merges, comes *closest* to weak-scaling ideality: when we multiply p by 20 (from 1 node $\times$ 32 PPN to 20 nodes $\times$ 32 PPN) the total data volume increases by the same factor, yet Circulant's run time increases by only $\approx 30\times$, whereas Bruck and Baseline slow down by $\approx 45\times$ and $\approx 280\times$, respectively. In other words, once inter-node communication dominates, the algorithm that minimises redundant data movement (Circulant) retains the highest weak-scaling efficiency, while the naïve gather-and-merge strategy collapses under the combined weight of network traffic and local merge cost.

4.1 Conclusion

In this project, we developed and analyzed three approaches for the parallel allgather-merge problem: a baseline method, a Bruck-based communication strategy, and a circulant allreduce variant. Our benchmarking results demonstrate that while the Baseline implementation suffers from poor scalability due to redundant communication and expensive p-way merges, the Bruck algorithm achieves notable improvements by integrating communication and merging. However, it is the Circulant allreduce-based approach that delivers the best weak-scaling performance, minimizing both communication overhead and merge complexity through efficient two-way merges of already sorted data. Overall, our findings confirm that careful integration of communication patterns and merging strategies is crucial for achieving scalable performance in distributed sorting problems.

Table 1: Performance of algorithms (execution time in microseconds). Configuration with 1 Nodes × 1 PPN.

Algorithm	Type	Message size (number of elements)					
		1	10	100	1000	10000	100000
Baseline	0	2.46×10^{-1}	4.56×10^{-1}	2.82	2.64×10^{1}	2.73×10^{2}	2.74×10^{3}
	1	2.23×10^{-1}	4.54×10^{-1}	2.83	2.64×10^{1}	2.73×10^{2}	2.74×10^{3}
	2	2.28×10^{-1}	4.55×10^{-1}	2.84	2.71×10^{1}	2.74×10^{2}	2.77×10^{3}
Bruck	0	1.83×10^{-1}	2.03×10^{-1}	2.87×10^{-1}	1.61	1.56×10^{1}	2.40×10^{3}
	1	1.75×10^{-1}	2.00×10^{-1}	2.82×10^{-1}	1.62	1.56×10^{1}	2.35×10^{3}
	2	1.74×10^{-1}	2.04×10^{-1}	2.85×10^{-1}	2.80	1.57×10^{1}	2.33×10^{3}
Circulant	0	5.64×10^{-2}	5.52×10^{-2}	6.00×10^{-2}	1.78×10^{-1}	2.51	2.23×10^{1}
	1	5.56×10^{-2}	5.55×10^{-2}	6.28×10^{-2}	1.78×10^{-1}	1.62	2.21×10^{1}
	2	5.68×10^{-2}	5.51×10^{-2}	6.06×10^{-2}	1.77×10^{-1}	2.78	2.21×10^{1}

Table 2: Performance of algorithms (execution time in microseconds). Configuration with 1 Nodes × 10 PPN.

Algorithm	Type	Message size (number of elements)					
		1	10	100	1000	10000	100000
Baseline	0	5.99	2.74×10^{1}	6.18×10^{1}	4.23×10^{2}	4.00×10^{3}	4.26×10^{4}
	1	5.43	2.85×10^{1}	4.82×10^{1}	3.18×10^{2}	3.13×10^{3}	3.46×10^{4}
	2	6.32	1.05×10^{1}	4.42×10^{1}	3.24×10^{2}	3.18×10^{3}	3.46×10^{4}
Bruck	0	6.58	1.06×10^{1}	2.05×10^{1}	1.28×10^{2}	1.30×10^{3}	3.13×10^{4}
	1	6.86	1.10×10^{1}	2.12×10^{1}	1.35×10^{2}	1.36×10^{3}	3.15×10^{4}
	2	7.32	1.08×10^{1}	2.10×10^{1}	1.37×10^{2}	1.36×10^{3}	3.05×10^{4}
Circulant	0	7.72	1.75×10^{1}	1.91×10^{1}	5.69×10^{1}	5.15×10^{2}	6.26×10^{3}
	1	6.63	1.50×10^{1}	2.10×10^{1}	6.45×10^{1}	5.82×10^{2}	6.65×10^{3}
	2	6.19	1.03×10^{1}	2.03×10^{1}	6.61×10^{1}	5.80×10^{2}	8.00×10^{3}

Table 3: Performance of algorithms (execution time in microseconds). Configuration with 1 Nodes × 32 PPN.

Algorithm	Type	Message size (number of elements)					
		1	10	100	1000	10000	100000
Baseline	0	1.60×10^{1}	4.64×10^{1}	1.86×10^{2}	1.59×10^{3}	1.56×10^{4}	1.59×10^{5}
	1	8.87	3.58×10^{1}	1.35×10^{2}	1.23×10^{3}	1.25×10^{4}	1.26×10^{5}
	2	7.66	2.69×10^{1}	2.05×10^{2}	1.91×10^{3}	1.88×10^{4}	1.87×10^{5}
Bruck	0	1.80×10^{1}	2.68×10^{1}	5.56×10^{1}	3.16×10^{2}	9.50×10^{3}	1.41×10^{5}
	1	1.21×10^{1}	1.44×10^{1}	5.21×10^{1}	3.20×10^{2}	9.55×10^{3}	1.41×10^{5}
	2	1.10×10^{1}	1.59×10^{1}	5.12×10^{1}	3.19×10^{2}	9.57×10^{3}	1.41×10^{5}
Circulant	0	1.03×10^{1}	1.39×10^{1}	3.15×10^{1}	2.04×10^{2}	2.42×10^{3}	3.79×10^{4}
	1	8.38	1.23×10^{1}	3.62×10^{1}	2.34×10^{2}	2.80×10^{3}	4.03×10^{4}
	2	7.88	1.03×10^{1}	3.64×10^{1}	2.32×10^{2}	2.80×10^{3}	4.09×10^{4}

Table 4: Performance of algorithms (execution time in microseconds). Configuration with 10 Nodes × 1 PPN.

Algorithm	**Type**	**Message size (number of elements)**					
		1	**10**	**100**	**1000**	**10000**	**100000**
Baseline	0	8.79	1.36×10^{1}	4.85×10^{1}	4.00×10^{2}	4.06×10^{3}	4.00×10^{4}
	1	9.14	1.26×10^{1}	3.88×10^{1}	3.12×10^{2}	3.19×10^{3}	3.14×10^{4}
	2	8.94	1.25×10^{1}	3.98×10^{1}	3.17×10^{2}	3.24×10^{3}	3.16×10^{4}
Bruck	0	8.59	1.13×10^{1}	1.88×10^{1}	1.34×10^{2}	1.26×10^{3}	2.28×10^{4}
	1	9.31	1.03×10^{1}	1.98×10^{1}	1.27×10^{2}	1.45×10^{3}	2.35×10^{4}
	2	9.55	1.04×10^{1}	2.03×10^{1}	1.25×10^{2}	1.43×10^{3}	2.35×10^{4}
Circulant	0	9.81	9.74	1.36×10^{1}	5.54×10^{1}	4.96×10^{2}	4.57×10^{3}
	1	8.59	1.12×10^{1}	1.50×10^{1}	6.54×10^{1}	5.78×10^{2}	5.20×10^{3}
	2	8.94	1.03×10^{1}	1.48×10^{1}	6.72×10^{1}	5.80×10^{2}	5.24×10^{3}

Table 5: Performance of algorithms (execution time in microseconds). Configuration with 10 Nodes × 10 PPN.

Algorithm	**Type**	**Message size (number of elements)**					
		1	**10**	**100**	**1000**	**10000**	**100000**
Baseline	0	2.21×10^{1}	8.77×10^{1}	5.90×10^{2}	5.47×10^{3}	5.59×10^{4}	5.68×10^{5}
	1	2.44×10^{1}	7.70×10^{1}	4.80×10^{2}	4.17×10^{3}	4.99×10^{4}	5.13×10^{5}
	2	2.62×10^{1}	9.94×10^{1}	8.01×10^{2}	7.82×10^{3}	8.09×10^{4}	7.99×10^{5}
Bruck	0	2.46×10^{1}	3.84×10^{1}	1.76×10^{2}	1.62×10^{3}	3.19×10^{4}	2.74×10^{5}
	1	2.04×10^{1}	4.12×10^{1}	1.81×10^{2}	1.71×10^{3}	3.15×10^{4}	2.73×10^{5}
	2	2.02×10^{1}	3.34×10^{1}	1.83×10^{2}	1.66×10^{3}	3.16×10^{4}	2.73×10^{5}
Circulant	0	2.15×10^{1}	3.20×10^{1}	1.17×10^{2}	8.50×10^{2}	9.36×10^{3}	1.17×10^{5}
	1	3.33×10^{1}	6.01×10^{1}	1.30×10^{2}	9.29×10^{2}	9.57×10^{3}	1.25×10^{5}
	2	1.73×10^{1}	7.74×10^{1}	1.15×10^{2}	9.23×10^{2}	9.55×10^{3}	1.16×10^{5}

Table 6: Performance of algorithms (execution time in microseconds). Configuration with 10 Nodes × 32 PPN.

Algorithm	**Type**	**Message size (number of elements)**					
		1	**10**	**100**	**1000**	**10000**	**100000**
Baseline	0	5.44×10^{1}	3.40×10^{2}	2.17×10^{3}	2.14×10^{4}	2.09×10^{5}	2.11×10^{6}
	1	4.13×10^{1}	2.85×10^{2}	1.94×10^{3}	1.98×10^{4}	2.20×10^{5}	2.11×10^{6}
	2	4.87×10^{1}	3.36×10^{2}	3.08×10^{3}	3.21×10^{4}	3.14×10^{5}	3.18×10^{6}
Bruck	0	3.05×10^{1}	9.66×10^{1}	7.74×10^{2}	1.25×10^{4}	1.64×10^{5}	1.66×10^{6}
	1	3.29×10^{1}	1.03×10^{2}	7.73×10^{2}	1.26×10^{4}	1.64×10^{5}	1.66×10^{6}
	2	3.20×10^{1}	2.20×10^{2}	7.78×10^{2}	1.26×10^{4}	1.65×10^{5}	1.66×10^{6}
Circulant	0	3.13×10^{1}	6.79×10^{1}	6.84×10^{2}	3.69×10^{3}	4.82×10^{4}	5.62×10^{5}
	1	2.80×10^{1}	6.61×10^{1}	5.43×10^{2}	4.21×10^{3}	5.02×10^{4}	5.82×10^{5}
	2	3.02×10^{1}	6.65×10^{1}	4.33×10^{2}	4.19×10^{3}	5.02×10^{4}	5.80×10^{5}

Table 7: Performance of algorithms (execution time in microseconds). Configuration with 20 Nodes × 1 PPN.

Algorithm	Type	Message size (number of elements)					
		1	10	100	1000	10000	100000
Baseline	0	1.60×10^{1}	2.15×10^{1}	9.86×10^{1}	8.95×10^{2}	8.88×10^{3}	8.77×10^{4}
	1	1.34×10^{1}	2.06×10^{1}	7.99×10^{1}	7.00×10^{2}	6.94×10^{3}	6.81×10^{4}
	2	1.33×10^{1}	1.94×10^{1}	1.01×10^{2}	8.75×10^{2}	8.91×10^{3}	8.76×10^{4}
Bruck	0	1.26×10^{1}	1.44×10^{1}	3.26×10^{1}	2.56×10^{2}	2.52×10^{3}	4.57×10^{4}
	1	1.25×10^{1}	1.63×10^{1}	3.50×10^{1}	2.63×10^{2}	2.66×10^{3}	4.74×10^{4}
	2	1.21×10^{1}	1.45×10^{1}	3.58×10^{1}	2.99×10^{2}	2.66×10^{3}	4.67×10^{4}
Circulant	0	1.23×10^{1}	1.36×10^{1}	2.52×10^{1}	1.26×10^{2}	1.24×10^{3}	1.07×10^{4}
	1	1.17×10^{1}	1.39×10^{1}	2.73×10^{1}	1.44×10^{2}	1.45×10^{3}	1.18×10^{4}
	2	1.16×10^{1}	1.30×10^{1}	2.56×10^{1}	1.41×10^{2}	1.46×10^{3}	1.18×10^{4}

Table 8: Performance of algorithms (execution time in microseconds). Configuration with 20 Nodes × 10 PPN.

Algorithm	Type	Message size (number of elements)					
		1	10	100	1000	10000	100000
Baseline	0	3.59×10^{1}	1.80×10^{2}	1.26×10^{3}	1.20×10^{4}	1.22×10^{5}	1.22×10^{6}
	1	3.85×10^{1}	1.22×10^{2}	9.02×10^{2}	8.80×10^{3}	1.10×10^{5}	1.06×10^{6}
	2	3.87×10^{1}	2.04×10^{2}	1.85×10^{3}	1.79×10^{4}	1.81×10^{5}	1.78×10^{6}
Bruck	0	2.93×10^{1}	5.94×10^{1}	3.47×10^{2}	3.55×10^{3}	6.15×10^{4}	5.56×10^{5}
	1	3.04×10^{1}	5.94×10^{1}	3.79×10^{2}	3.67×10^{3}	6.06×10^{4}	5.59×10^{5}
	2	2.75×10^{1}	6.27×10^{1}	3.74×10^{2}	3.65×10^{3}	6.11×10^{4}	5.60×10^{5}
Circulant	0	2.65×10^{1}	4.53×10^{1}	2.09×10^{2}	1.72×10^{3}	1.97×10^{4}	2.38×10^{5}
	1	2.49×10^{1}	4.98×10^{1}	2.29×10^{2}	1.95×10^{3}	2.17×10^{4}	2.46×10^{5}
	2	2.59×10^{2}	9.93×10^{1}	3.74×10^{2}	1.97×10^{3}	2.14×10^{4}	2.46×10^{5}

Table 9: Performance of algorithms (execution time in microseconds). Configuration with 20 Nodes × 32 PPN.

Algorithm	Type	Message size (number of elements)					
		1	10	100	1000	10000	100000
Baseline	0	1.43×10^{2}	5.80×10^{2}	4.98×10^{3}	4.79×10^{4}	4.48×10^{5}	4.46×10^{6}
	1	9.16×10^{1}	5.82×10^{2}	5.18×10^{3}	5.74×10^{4}	5.51×10^{5}	5.35×10^{6}
	2	2.49×10^{2}	7.23×10^{2}	6.97×10^{3}	7.35×10^{4}	7.06×10^{5}	7.20×10^{6}
Bruck	0	4.00×10^{1}	1.81×10^{2}	1.62×10^{3}	2.67×10^{4}	3.41×10^{5}	3.36×10^{6}
	1	6.19×10^{1}	1.92×10^{2}	1.68×10^{3}	2.68×10^{4}	3.43×10^{5}	3.33×10^{6}
	2	4.37×10^{1}	4.13×10^{2}	1.75×10^{3}	2.71×10^{4}	3.40×10^{5}	3.33×10^{6}
Circulant	0	6.24×10^{1}	1.13×10^{2}	7.80×10^{2}	9.09×10^{3}	1.17×10^{5}	1.14×10^{6}
	1	2.72×10^{2}	2.84×10^{2}	8.81×10^{2}	9.40×10^{3}	1.24×10^{5}	1.20×10^{6}
	2	3.65×10^{1}	1.13×10^{2}	8.82×10^{2}	9.56×10^{3}	1.24×10^{5}	1.20×10^{6}

Appendix

Algorithm 7 Merge Two Sorted Arrays (std::merge)

```
1:  procedure MERGE(A, B, C)
2:      i ← 0                                   ▷ Index for A
3:      j ← 0                                   ▷ Index for B
4:      k ← 0                                   ▷ Index for C
5:      while i < length(A) and j < length(B) do
6:          if A[i] ≤ B[j] then
7:              C[k] ← A[i]
8:              i ← i + 1
9:          else
10:             C[k] ← B[j]
11:             j ← j + 1
12:         end if
13:         k ← k + 1
14:     end while
15:     while i < length(A) do
16:         C[k] ← A[i]
17:         i, k ← i + 1, k + 1
18:     end while
19:     while j < length(B) do
20:         C[k] ← B[j]
21:         j, k ← j + 1, k + 1
22:     end while
23: end procedure
```

Data Structure: `HeapNode`

`idx`
: Integer. Index of the source array.

`pos`
: Integer. Position of the element within its corresponding block.

`ptr`
: Pointer to the current element in memory (used for direct access in buffer).

Algorithm 8 P-Way Merge Using Min-Heap

```
1: procedure P_Way_Merge(W, m, p)
2:     for i = 0 to p − 1 do
3:         Heap[i] ← Node(i, 0, pointer to element at index i · m)
4:     end for
5:     positions[0 . . . p − 1] ← 0
6:     output[0 . . . p · m − 1]
7:     for out_idx = 0 to p · m − 1 do
8:         if Heap is empty then
9:             break
10:        end if
11:        Pop MinNode from Heap
12:        output[out_idx] ← MinNode.ptr
13:        positions[MinNode.idx] ← positions[MinNode.idx] + 1
14:        if positions[MinNode.idx] < m then
15:            Push NextNode into Heap
16:        end if
17:    end for
18:    W ← output
19: end procedure
```

References

[1] Jesper Larsson Träff and Sascha Hunold. High performance computing 2025s: Assignment 1 – allgather-merge, 2025. Assignment for the course High Performance Computing, Summer term 2025.

[2] Research Group for Parallel Computing (Institute for Computer Engineering) TU Wien. Hydra cluster documentation, n.d. Accessed: 2025-06-02.

[3] Wikipedia contributors. K-way merge algorithm. `https://en.wikipedia.org/wiki/K-way_merge_algorithm`, n.d. Accessed: 2025-06-02.

[4] Felix Wolf. External merge sort with loser trees. YouTube video, 2020. Accessed: 2025-06-02.